

Smart Ticket Queue

A Complete Beginner's Guide to Background Jobs and AI APIs

Author: Shamil Bin Hossain | August 2026

Chapter 1: The Core Concept

Welcome to the complete book for the **Smart Ticket Queue (AsyncTriage API)** project! If you are a beginner, this book will explain everything from start to finish in simple, easy-to-understand English.

The Restaurant Analogy (What did we build?)

Imagine a very busy restaurant. A customer walks in, orders a burger, and waits at the counter. The waiter goes to the kitchen, cooks the burger, and brings it back 10 minutes later. Meanwhile, a huge line of angry customers forms behind the first person because the waiter is busy cooking instead of taking orders. This is called a **Blocking Task**.

In our previous project, our API acted exactly like this bad waiter. When a user sent a message, our API stopped everything to wait for the Artificial Intelligence (AI) to think and respond. If 100 users messaged at once, the server would crash.

The Smart Solution

We fixed this by hiring a Chef and a Ticket Board. Now, when a customer orders a burger:

1. The Waiter (**FastAPI**) takes the order instantly, gives the customer a Token Number (**Job ID**), and says "Please have a seat, your order is processing." This takes less than 1 second! (This is our `POST 202 Accepted` route).
2. The Waiter puts the order on a Ticket Board (**Redis**).
3. The Chef (**Celery Worker**) looks at the board, takes the ticket, and cooks the burger (calls the AI) in the background.
4. The customer can check the Token Board (**Status Endpoint**) at any time to see if their food is ready.

This is called an **Asynchronous Background Job System**, and it is how massive companies like YouTube, Facebook, and Netflix handle heavy tasks!

Chapter 2: Understanding The Servers

To make this magic happen, our project uses three different servers working together simultaneously.

- **FastAPI (The Waiter):** This is our main web server. It handles the internet traffic. When a user sends a POST request with a ticket, or a GET request to check the status, FastAPI handles it at lightning speed.
- **Redis (The Ticket Board / Message Broker):** Redis is a super-fast, in-memory database. We use it as a "Queue" (a waiting line). When FastAPI receives a message, it drops it into Redis. Redis holds onto the message safely until someone is ready to process it.
- **Celery (The Chef / Worker):** Celery is a separate program running in the background. Its only job is to constantly watch Redis. As soon as a new message appears in Redis, Celery grabs it, contacts the AI, gets the result, and saves the final result back into Redis so FastAPI can show it to the user.

Chapter 3: The AI Prompts & Explanations

We built this entire system by giving precise instructions (prompts) to an AI coding agent named **Antigravity AI**. Here is exactly what we asked it to do and why.

Prompt 1: Project Setup & Foundation

Act as an expert Python backend engineer. We are starting a new project called 'AsyncTriage API'. Let's set up the foundational files. Please execute the following tasks:

1. Create a `requirements.txt` file containing exactly these packages: `fastapi`, `uvicorn`, `celery`, `redis`, `openai`, `pydantic`, and `python-dotenv`.
2. Create a `.env.example` file with two variables: `LLM\_API\_KEY=your\_api\_key\_here` and `LLM\_STUB=1`.
3. Create a folder named `prompts` and inside it, `triage-v1.md`. Write a detailed System Prompt for a Customer Support Triage AI (Extract Category, Urgency, Summary, action_required).
4. Create `src/llm/schema.py` and write a Pydantic model named `TicketAnalysisOutput` to validate the AI's JSON output.

Why we did this: Before building the complex background jobs, we needed a foundation. We installed the necessary tools (requirements), created a safe place for our secret passwords (.env), gave the AI its brain/instructions (triage-v1.md), and created a strict blueprint (schema.py) to make sure the AI always answers in the exact format we want.

Prompt 2: Building the Background Worker

Act as an expert Python backend engineer. We are on Step 2.

1. Create a file named ``worker.py`` inside the ``src`` folder.
2. Initialize a Celery app connecting to a local Redis broker (``redis://localhost:6379/0``).
3. Create a Celery task named ``process_ticket_task`` that takes a ``ticket_text`` string as input.
4. Implement the LLM logic inside this task (load API key, read prompt, call AI, validate JSON).
5. VERY IMPORTANT: Add error handling and automatic retries (``max_retries=3``, ``retry_backoff=True``) in case the LLM API times out.

Why we did this: This is where the magic happens. We created our "Chef" (`worker.py`). We told Celery to connect to Redis so it knows where to look for tasks. The most important part was asking for **Automatic Retries**. AI servers can sometimes crash or freeze. Instead of failing the user's task, our system is smart enough to wait a few seconds and try again automatically!

Prompt 3: Building the API Endpoints

Act as an expert Python backend engineer. We are on Step 3.

1. Create ``src/routes/ticket.py``.
2. Implement ``POST /api/v1/triage``: It should accept the user's message, dispatch the job to Celery using ``process_ticket_task.delay()``, and immediately return an HTTP ``202 Accepted`` status with a ``job_id``.
3. Implement ``GET /api/v1/status/{job_id}``: Check the Celery task status. Return PENDING, SUCCESS, or FAILURE. If SUCCESS, include the AI's result.
4. Create ``src/main.py`` and connect everything.

Why we did this: We built the doors for our users to enter. The `delay()` function is the secret sauce. Instead of running the AI code immediately, `delay()` throws the job into Redis and instantly returns the Job ID to the user. The GET route simply looks at Redis to see if the Celery worker has finished the job yet.

Chapter 4: File Structure (What and Why)

A professional project must be organized. Here is a breakdown of every file created in this project and why it is absolutely necessary.

File / Folder Name	Purpose and Explanation
<code>.env</code>	Stores your secret <code>LLM_API_KEY</code> . This is your master password for the AI. It is NEVER uploaded to GitHub to prevent hackers from stealing your money.
<code>.env.example</code>	A fake version of the <code>.env</code> file. We upload this to GitHub so other developers know what variables they need to create their own <code>.env</code> file.
<code>requirements.txt</code>	A shopping list for Python. It contains all the tools needed (fastapi, celery, redis, etc.). Typing <code>pip install -r requirements.txt</code> installs everything automatically.
<code>prompts/triage-v1.md</code>	The instruction manual for the AI. It tells the AI to act like a customer support agent and how to categorize tickets. Keeping it in a Markdown file makes it easy to read and edit.
<code>src/llm/schema.py</code>	Contains "Pydantic Models". Think of this as a strict mold or blueprint. It forces the AI to reply with exactly 4 things: Category, Urgency, Summary, and action_required. If the AI makes a mistake, this file catches it.
<code>src/worker.py</code>	The Chef. This file runs entirely in the background using Celery. It does the heavy lifting: talking to OpenAI, waiting for the response, and handling errors and retries.
<code>src/routes/ticket.py</code>	The Waiter. It contains the web addresses (endpoints). It has the <code>POST</code> route to accept new tickets and give Job IDs, and the <code>GET</code> route to check task statuses.
<code>src/main.py</code>	

The Manager. This is the entry point of the app. It starts the FastAPI server and connects the routes from `ticket.py` so the app runs as a single unit.

Chapter 5: How to Run and Test the Project

Because this is a complex background job system, you cannot just click "Run" once. You must start the three servers independently. Here is the step-by-step guide to running and testing your code like a pro.

Step 1: Start Redis (The Queue)

Open a terminal and make sure Redis is running on your Mac. You only have to do this once.

```
brew services start redis
```

Step 2: Start the Celery Worker (Terminal 1)

Open a new terminal window inside your project folder. Start the background worker. It will stay open and watch for new jobs.

```
celery -A src.worker.celery_app worker --loglevel=info
```

Step 3: Start the FastAPI Web Server (Terminal 2)

Open a second terminal window. Start your main web API.

```
uvicorn src.main:app --reload
```

Step 4: Send a Task (Test 1 - POST)

Now, open a third terminal. Pretend you are a user sending a complaint about a billing issue. Run this cURL command:

```
curl -X POST "http://127.0.0.1:8000/api/v1/triage" -H "Content-Type: application/json" -d '{"ticket_text": "I was charged twice for my subscription this month!"}'
```

What happens: The API responds instantly (in milliseconds) with something like: `{"job_id": "abc-123-xyz", "message": "Ticket processing started..."}`

Step 5: Check the Status (Test 2 - GET)

Copy the `job_id` you just received. Use it in the GET command to see if the AI has finished reading the message.

```
curl -X GET "http://127.0.0.1:8000/api/v1/status/abc-123-xyz"
```

What you will see:

- If it is still working, it will say: `"status": "PENDING"`
- If the AI crashed but is trying again, it will say: `"status": "RETRY"`
- If it is finished, it will say: `"status": "SUCCESS"` and it will show you the beautifully formatted JSON with `"category": "BILLING"` and `"urgency": "HIGH"`.

Conclusion

Congratulations! You have successfully built a real-world, highly scalable architecture. By decoupling (separating) the fast web requests from the slow AI processing, you have ensured that your API will never crash under heavy load. You are now officially engineering like the pros!